Technical Analysis of Cobalt Strike Beacon Detection Strategies Across Endpoint and Network Telemetry (2025–2026)

Executive Summary

The persistence of Cobalt Strike as a primary post-exploitation framework into 2026 remains a central challenge for the cybersecurity industry, despite high-profile law enforcement actions such as Operation Morpheus in 2024, which initially disrupted nearly 600 malicious servers.[1] The framework's resilience is rooted in its "evasion through flexibility" philosophy, which has allowed threat actors—ranging from state-sponsored APTs to sophisticated ransomware collectives—to adapt to advancing security controls.[2] The release of Cobalt Strike version 4.12 in late 2025 has further modernized the toolkit, introducing a comprehensive REST API for automation and "drip-loading" capabilities that fundamentally alter the memory footprint of the Beacon agent.[3]

Detection of modern Beacon activity requires a pivot away from static indicators toward high-fidelity behavioral analytics. On the endpoint, the emergence of User-Defined Reflective Loaders (UDRLs) like "Eden" and "Lucky Strike" necessitates that defenders scrutinize memory integrity and the legitimacy of call stacks rather than relying on file-based signatures.[5] In the network domain, the transition from the JA3 TLS fingerprinting standard to the more robust and modular JA4+ suite in 2025–2026 has provided analysts with more stable identifiers for identifying malicious infrastructure, even when traffic is fully encrypted.[6] This report synthesizes multi-source telemetry from leading vendors like Zscaler, Elastic, and Mandiant to provide a comprehensive detection blueprint for the modern threat landscape.

Cobalt Strike Beacon Tradecraft Overview

The technical architecture of Cobalt Strike has evolved significantly between 2024 and 2026. While the core "Beacon" agent remains a staged or stageless DLL designed for persistent command-and-control, the methods by which it is delivered and operated have become increasingly modular.[8]

Modern Beacon Communication and Protocols

Cobalt Strike supports a variety of communication channels, including HTTP/S, DNS, SMB, and custom User-Defined C2 (UDC2) channels.[10] In 2025, the utilization of alternative data channels has grown, with adversaries abusing legitimate cloud services like Azure Service Bus for egress

traffic.[5]

Protocol	Communication Mechanism	Common Evasion in 2025–2026
HTTP/HTTPS	Traditional GET/POST requests mimicking standard web traffic.	Abuse of CDN fronting and cloud-integrated redirectors (e.g., Cloudflare Workers).[10]
DNS	Data exfiltration via A, AAAA, or TXT record queries.	Sophisticated sub-domain patterns mimicking legitimate cloud API calls.[13]
SMB	Peer-to-peer communication over named pipes for lateral movement.	Randomized pipe names and access patterns mimicking standard Windows IPC.[1]
UDC2	Custom protocols (e.g., ICMP) implemented via Beacon Object Files (BOFs).	Encapsulating C2 within non-standard protocols to bypass simple proxy filters.[11]

The Malleable C2 profile continues to be the primary mechanism for defining these behaviors. Profiles such as "jQuery" remain popular due to their ability to blend into enterprise environments, though many actors now utilize automated profile generators like "SourcePoint" to ensure that every deployment has a unique network and memory signature.[2]

Fileless Execution and In-Memory Techniques

The shift toward fileless execution is nearly complete in high-maturity operations. Instead of dropping a traditional executable to disk, attackers utilize "loaders"—often written in languages like Rust or Nim—to inject the Beacon into a legitimate process's memory space.[12] The introduction of "drip-loading" in version 4.12 allows the Beacon to allocate memory incrementally, avoiding the "all-at-once" memory allocation spikes that trigger modern EDR sensors.[3]

The process of loading the Beacon reflectively involves several critical stages, each of which presents a potential detection opportunity:

1. **Memory Allocation:** The loader claims a region of memory (typically RW) using VirtualAlloc or NtAllocateVirtualMemory.[16]

2. **Payload Mapping:** The Beacon's PE sections are copied into the allocated space.
3. **Relocation and Import Resolution:** The loader manually resolves function pointers for necessary system DLLs (e.g., kernel32.dll).[2]
4. **Permission Transition:** The memory is transitioned to RX (Read-Execute) before execution.[17]

Post-Exploitation Behavior and Lateral Movement

Once a Beacon session is established, attackers utilize it to perform a variety of post-exploitation tasks. As of 2025, the use of "Living-off-the-Land" (LotL) techniques is the dominant method for lateral movement and credential access, with cmd.exe (48.7%) and PowerShell (47.3%) leading tool usage among APT groups.[18]

Post-Exploitation Task	Common Tools Used (2025–2026)	Forensic Artifacts
Credential Access	Mimikatz, Brute Ratel, tailored BOFs.	LSASS process access, unusual registry modifications (HKLM\SAM).
Lateral Movement	PsExec, WinRM, WMI, SMB Named Pipes.	Event ID 7045 (Service Installation), Event ID 4624 (Logon).
Persistence	Task Scheduler, Registry Run Keys, WMI Event Subscriptions.	Scheduled Task creation events, suspicious binary paths in Run keys.
Reconnaissance	net.exe, ipconfig, tasklist, Custom BOFs.	Rapid execution of enumeration commands from a single process.

The move toward modularity means that many of these tasks are now performed via BOFs (Beacon Object Files), which execute in the Beacon's memory space and then exit, leaving no

file-based trace and minimal process telemetry.[2]

EDR Detection Techniques

Endpoint Detection and Response (EDR) systems have matured significantly, moving beyond simple signature matching to analyze the fundamental behaviors of a process. For detection engineers, the goal is to identify the "unnatural" artifacts created by the Beacon's presence in memory and its interaction with the operating system.[16]

Memory Artifacts and Integrity Scanning

The most reliable indicator of a reflectively loaded Beacon is the presence of executable code in "unbacked" memory regions—pages that are not associated with a physical file on disk (i.e., MEM_PRIVATE). In a standard Windows environment, almost all executable code resides in MEM_IMAGE sections, representing legitimately loaded DLLs or EXEs.[16]

Detection strategies for memory-resident Beacons include:

- **Permissions Analysis:** Identifying memory regions that transition from RW to RX or those that remain RWX. Sophisticated profiles now set userwx "false" to avoid the RWX red flag, but the transition to RX is still a high-signal event when originating from unbacked memory.[17]
- **Header Stomping Identification:** While the stomppe setting overwrites the MZ header and PE signature to evade simple scanners, this very act is suspicious. Scanners that look for PE-like structures (e.g., section names like .text, .rdata) within "stomped" regions can reliably identify hidden payloads.[17]
- **Sleep Mask Signatures:** The sleep_mask setting encrypts the Beacon's memory during its sleep cycle. However, the mask code itself—responsible for the decryption—must remain executable and unencrypted. Threat hunters can utilize YARA rules targeting the static byte sequences of popular sleep mask implementations, such as "Sleepmask-VS".[2]

The mathematical probability of a detection event $P(D)$ during a memory scan can be calculated based on the scan frequency f and the Beacon's active window w (when it is unmasked and processing tasks):

$$P(D) = 1 - (1 - \frac{w}{T})^n$$

where T is the total cycle time (sleep + jitter) and n is the number of scans. This highlights the importance of trigger-based scanning—performing a memory scan immediately following a suspicious API call or network connection—rather than relying on periodic sweeps.[16]

Process Injection and Parent-Child Anomalies

Despite the focus on fileless execution, the Beacon still needs to exist within a process context. Defenders should monitor for suspicious parent-child relationships and injection-related API calls.[1]

Common anomalies in 2025–2026 include:

- **Suspicious Spawnto Targets:** The spawnto_x64 profile setting defines which process the Beacon will use for post-exploitation jobs. Common targets include wmiprvse.exe and searchprotocolhost.exe.[17] An alert should be generated when these processes are spawned with an empty command line or by an unusual parent process like rundll32.exe.[1]
- **Process Hollowing and Module Stomping:** Monitor for the CREATE_SUSPENDED flag in CreateProcess followed by memory manipulation APIs like NtUnmapViewOfSection or VirtualAllocEx.[19]
- **Thread Context Manipulation:** The use of SetThreadContext or NtQueueApcThread to redirect a legitimate thread's execution to malicious shellcode is a high-confidence indicator of injection.[19]

Named Pipe Monitoring for SMB Beacons

Cobalt Strike's SMB Beacon relies on named pipes for peer-to-peer communication. While default names like \pipe\msagent_ are well-known, modern attackers randomize these strings. Detection must focus on behavioral heuristics [1]:

- **Long-Tail Analysis of Pipe Names:** Identifying named pipes that appear infrequently (Low Frequency Observation - LFO) across the environment.
- **Cross-Process Pipe Communication:** Monitoring for processes that connect to named pipes created by "unbacked" or suspicious processes.[10]
- **Pipe Permissions:** Beacons often create pipes with permissive security descriptors to facilitate communication from multiple compromised hosts.

Web Proxy and Network Detection Techniques

The network remains a critical visibility gap for many organizations, especially as encryption becomes the default for almost all C2 traffic. However, the metadata of the TLS handshake and the behavioral patterns of the connection provide significant detection opportunities.[6]

The Transition to JA4+ Fingerprinting

The JA3 standard, introduced in 2017, was a breakthrough in TLS fingerprinting but suffered from stability issues due to its reliance on the exact order of TLS extensions. In 2025–2026, the industry has universally adopted the JA4+ suite, which provides a modular and more resilient fingerprinting standard.[6]

A JA4 fingerprint consists of three segments (a_b_c):

1. **Segment A (Protocol):** Encodes the protocol version, presence of SNI, cipher/extension counts, and ALPN (e.g., t13d1516h2 for TLS 1.3 with SNI and HTTP/2).[6]
2. **Segment B (Cipher Hash):** A truncated hash of the sorted cipher suites.
3. **Segment C (Extension Hash):** A truncated hash of the sorted extensions.

Detection Scenario	JA4 Signal	Analyst Implication
Tool Identification	Specific JA4 hashes associated with frameworks like Cobalt Strike or Metasploit.[21]	The fingerprint serves as a direct indicator of offensive tooling, regardless of the domain name used.
User-Agent Mismatch	A JA4 fingerprint indicating a Python library (e.g., requests) while the HTTP User-Agent claims to be Chrome.[6]	This is a high-fidelity indicator of automation or malicious spoofing.
Infrastructure Clustering	Multiple disparate domains sharing the same JA4 fingerprint.	Allows analysts to pivot from a single IOC to an entire C2 infrastructure.

Beaconing Patterns and Jitter Analysis

The periodic "callback" is a fundamental requirement for asynchronous C2. Detection engineers can identify this behavior by analyzing the inter-arrival time of packets.[10]

Legitimate human traffic is characterized by "burstiness"—high volume during active browsing followed by long periods of inactivity. In contrast, Beacon traffic follows a schedule defined by the sleep and jitter variables. For example, a 60-second sleep with 20% jitter will result in connections occurring at intervals between 48 and 72 seconds.[17]

By applying a Fourier Transform or similar statistical analysis to network flow logs, defenders can identify these hidden periodicities even when jitter is applied. In 2026, many Zscaler-like proxy solutions have integrated AI-powered detection for unknown or suspicious C2 traffic that specifically alerts on these rhythmic patterns.[24]

DNS Anomalies and DGA-Like Behavior

DNS Beacons utilize the DNS protocol to tunnel data, often hiding within A-record or TXT-record queries. Indicators of DNS-based C2 include [1]:

- **Subdomain Entropy:** High-entropy subdomains (e.g., a1b2c3d4.c2server.com) indicating

encoded data.

- **Query Frequency:** A sudden spike in DNS queries for a single parent domain, especially from a single host.
- **Suspicious Responses:** Beacons may return specific IP addresses, such as 0.0.0.0, to signal that the Team Server is busy or that the tasking has completed.[1]
- **Long-Tail Domain Analysis:** Monitoring for queries to newly registered domains (NRDs) or domains with very low global popularity.[13]

Cross-Telemetry Correlation Strategies

The most effective detections are those that combine signals from both the endpoint and the network to build a "behavioral chain." This multi-signal approach significantly reduces false positives and allows for the detection of highly stealthy implants.[10]

Mapping Endpoint Activity to Outbound C2 Traffic

A high-fidelity alert can be generated when an "unbacked" process (identified by EDR) initiates a network connection that matches a known C2 pattern (identified by proxy logs).

Step in Attack Chain	Telemetry Source	Signal / Event
Injection	EDR	Suspicious VirtualProtect call on MEM_PRIVATE memory.
Persistence	EDR	New Scheduled Task created pointing to rundll32.exe.
Command & Control	Proxy/NDR	HTTPS connection to an NRD with a suspicious JA4 fingerprint.
Data Exfiltration	Proxy/NDR	Periodic, low-volume POST requests to a cloud-fronted URI.

By correlating these events using a shared ProcessID or HostID within a SIEM or XDR platform, analysts can visualize the full scope of an intrusion.[26]

Multi-Signal Detection (Behavioral Chaining)

"Behavioral Chaining" involves looking for sequences of events that are individually benign but collectively malicious. A common chain seen in 2025 ransomware intrusions involves:

1. **Initial Access:** A "ClickFix" social engineering event where a user is tricked into running a PowerShell command from a compromised website.[29]
2. **Staging:** PowerShell execution leads to the download of a GOSHELL loader.[30]
3. **Execution:** GOSHELL reflectively loads a Cobalt Strike Beacon into wmiprvse.exe.[17]
4. **Reconnaissance:** The wmiprvse.exe process executes net group "Domain Admins" /domain.[1]

A detection rule that triggers on any of these steps might have a high false-positive rate, but a rule that triggers on the *sequence* of these events is a near-certain indicator of a breach.[10]

Detection Engineering Playbook

This playbook provides actionable logic for implementing Cobalt Strike detections in modern security environments.

Sample Queries and Detection Logic

KQL (Microsoft Sentinel / Defender): Detecting Unbacked Executable Memory

Code snippet

```
DeviceMemoryEvents

| where ActionType == "MemoryPermissionsChanged"
| where NewPermissions == "ExecuteRead"
| where MemoryType == "Private" or MemoryType == "None"
| where not(InitiatingProcessFileName in~ ("msmpeng.exe", "searchindexer.exe"))
| project TimeGenerated, DeviceName, InitiatingProcessFileName,
InitiatingProcessCommandLine, NewPermissions
```

[16]

Sigma Rule: Suspicious Cobalt Strike DNS Patterns

YAML

```yaml
title: Suspicious Cobalt Strike DNS Beaconing - Sysmon
id: f356a9c4-effd-4608-bbf8-408afd5cd006
status: test
description: Detects a program that invoked suspicious DNS queries known from Cobalt Strike beacons
logsource:
  category: dns_query
  product: windows
detection:
  selection1:
    QueryName|startswith:
      - 'aaa.stage.'
      - 'post.1'
  selection2:
    QueryName|contains: '.stage.123456.'
  condition: 1 of selection*
level: critical
```

13

Sigma Rule: Lateral Movement via Malicious Service Installation

YAML

```yaml
title: CobaltStrike Service Installations - System
id: 5a105d34-05fc-401e-8553-272b45c1522d
logsource:
  product: windows
  service: system
detection:
  selection_id:
    Provider_Name: 'Service Control Manager'
    EventID: 7045
  selection1:
    ImagePath|contains|all:
      - 'ADMIN$'
```

```
        - '.exe'
    condition: selection_id and selection1
level: critical
```

31

Heuristics for Operationalization

To improve the efficiency of a SOC, detections should be prioritized based on their fidelity and the cost of the false positives they generate.[27]

- **Heuristic 1 (High Fidelity):** Alert on any MEM_PRIVATE region with RX permissions that initiates a network connection to a domain with a reputation score below 50.
- **Heuristic 2 (Medium Fidelity):** Monitor for rundll32.exe or svchost.exe spawning a shell (cmd.exe or powershell.exe) where the parent process lacks the standard command-line arguments for that legitimate binary.[20]
- **Heuristic 3 (Threat Hunting):** Identify all JA4 fingerprints that have appeared for the first time in the last 24 hours and correlate them with high-volume outbound traffic.[6]

Evasion Techniques vs. Detection Countermeasures

As detection capabilities improve, the Cobalt Strike development team and the wider offensive community continue to innovate, specifically targeting the telemetry points used by EDR and NDR solutions.[2]

Indirect Syscalls and Call Stack Spoofing

Modern EDRs often hook high-level Windows APIs in ntdll.dll or kernel32.dll. To bypass these, Cobalt Strike (specifically through the "Sleepmask-VS" project) now utilizes **indirect syscalls**.[2] Instead of calling the API, the Beacon executes a syscall instruction found within the legitimate code of ntdll.dll, effectively bypassing the EDR's hook.

Countermeasure: Detection must move to the kernel level (e.g., using ETW-Ti). Defenders should monitor the CallStack during a system call. If the return address on the stack points to an "unbacked" memory region rather than a legitimate module, it is a high-signal indicator of a syscall bypass attempt.[2]

Advanced Memory Evasion (Eden and Drip-Loading)

The "Eden" project is a Proof-of-Concept UDRL that combines "page streaming" with a modular call gate (Draugr) to minimize the Beacon's presence in memory.[5] This is complemented by "drip-loading" in CS 4.12, which allows for the incremental allocation of memory, avoiding the "all-at-once" allocation signatures.[3]

Countermeasure: Traditional point-in-time memory scanning is insufficient against page streaming. Defenders must implement **integrity-based continuous monitoring** that tracks

the total volume of private executable memory over time and flags processes that exhibit "slow-growth" executable allocations without a corresponding image load.[3]

Protocol Mimicry and Domain Fronting

While domain fronting has become harder due to changes at major cloud providers, attackers have shifted to "domain blending" and the abuse of cloud redirectors like Cloudflare Workers.[10] These services allow the attacker to hide their Team Server behind a legitimate, high-reputation infrastructure.

Countermeasure: Proxy logs should be analyzed for **TLS/HTTP Metadata Inconsistencies**. For example, a request to a Cloudflare Worker that includes a Cookie or Header structure defined in a Malleable C2 profile but missing from standard web browser traffic to that same service. The JA4H fingerprint (HTTP client) is particularly effective here for identifying non-browser automation.[6]

Emerging Trends

The C2 landscape in 2026 is defined by the proliferation of modular, open-source frameworks and the integration of AI into both offensive and defensive operations.

The Shift to Alternate C2 Frameworks

While Cobalt Strike remains the leader, frameworks like **Sliver**, **Havoc**, and **Mythic** have seen significant adoption.[8] These tools often offer native cross-platform support (Linux, macOS) where EDR coverage is historically weaker.[1]

Framework	Primary Evasive Strength	Detection Focus
Sliver	Cross-platform, MTLS encryption, Go-based obfuscation.[32]	Go runtime artifacts, unusual DNS/HTTP egress from Linux servers.
Havoc	"Demon" agent with advanced stack spoofing and sleep masking.[34]	Indirect syscall detection, memory integrity scanning.
Mythic	Extremely modular with Python/JavaScript agents.[34]	Monitoring for suspicious scripting interpreter activity (Python, Node.js).

The Role of AI/ML in Detection Pipelines

Security vendors are increasingly utilizing AI to generate "proactive" decoys and to automate

the identification of C2 patterns.[36] Zscaler's 2026 AI Security Report highlights that enterprise AI activity increased by over 90% in 2025, providing a vast amount of telemetry for training models to distinguish between legitimate AI-driven applications and malicious automation.[30]

Changes in Tradecraft post-Operation Morpheus

Following the 2024 law enforcement disruption, attackers have moved toward more decentralized and ephemeral infrastructure. The use of "just-in-time" (JIT) compilation for payloads and the rotation of C2 domains every few hours has made static blacklisting obsolete.[1] Modern defense must focus on the "unchanging" behaviors of post-exploitation—credential dumping, lateral movement, and internal reconnaissance—rather than the initial delivery mechanism.[1]

Key Takeaways for Threat Hunters

A successful defense against Cobalt Strike in 2026 requires a layered approach that integrates endpoint, network, and cloud telemetry.

1. **Assume Memory Compromise:** Move beyond file-based signatures. Implement automated memory integrity scanning that targets "unbacked" executable memory and suspicious permission transitions.[16]
2. **Standardize on JA4+:** Upgrade network monitoring tools to support the JA4+ suite. Utilize the modularity of JA4 to cluster malicious infrastructure and identify User-Agent inconsistencies.[6]
3. **Monitor the LotL Baseline:** Defenders must have a clear understanding of "normal" administrative activity. The presence of net.exe, psexec, or wmic should be alerted upon if it originates from non-administrative accounts or unusual parent processes.[1]
4. **Operationalize Behavioral Chaining:** Build detection logic that correlates initial access (e.g., ClickFix) with post-exploitation behaviors (e.g., memory injection) and network egress (e.g., periodic beaconing).[10]
5. **Focus on Linux and Cloud Gaps:** As attackers migrate to frameworks like CrossC2 and Sliver to target non-Windows assets, ensure that EDR and network visibility extend to your Linux servers and cloud-native workloads.[8]

Works cited

1. Cobalt Strike Detection & Defense Guide - Vectra AI, accessed April 4, 2026, https://www.vectra.ai/topics/cobalt-strike
2. EDR evasion and Cobalt Strike - S4 Applications, accessed April 4, 2026, https://s4applications.uk/edr-evasion-and-cobalt-strike/
3. Cobalt Strike Release Notes, accessed April 4, 2026, https://download.cobaltstrike.com/releasenotes.txt
4. Release Out: Finally, Some REST | Cobalt Strike Blog, accessed April 4, 2026, https://www.cobaltstrike.com/blog/release-out-finally-some-rest

5. Introducing Cobalt Strike Research Labs, accessed April 4, 2026, https://www.cobaltstrike.com/blog/introducing-cobalt-strike-research-labs
6. TLS Fingerprinting Guide 2026 | JA4+ Detection - Proxies.sx, accessed April 4, 2026, https://www.proxies.sx/use-cases/privacy/tls-fingerprint
7. GitHub - FoxIO-LLC/ja4: JA4+ is a suite of network fingerprinting standards, accessed April 4, 2026, https://github.com/FoxIO-LLC/ja4
8. Modular C2 Frameworks Quietly Redefine Threat Operations for ..., accessed April 4, 2026, https://blog.alphahunt.io/modular-c2-frameworks-quietly-redefine-threat-operations-for-2025-2026/
9. Defining Cobalt Strike Components & BEACON | Google Cloud Blog, accessed April 4, 2026, https://cloud.google.com/blog/topics/threat-intelligence/defining-cobalt-strike-components
10. Top strategies to detect and prevent cobalt strike - Fidelis Security, accessed April 4, 2026, https://fidelissecurity.com/threatgeek/threat-detection-response/cobalt-strike-detection/
11. Cobalt Strike · GitHub, accessed April 4, 2026, https://github.com/cobalt-strike
12. SloppyLemming Deploys BurrowShell and Rust-Based RAT to Target Pakistan and Bangladesh - Arctic Wolf, accessed April 4, 2026, https://arcticwolf.com/resources/blog/sloppylemming-deploys-burrowshell-and-rust-based-rat-to-target-pakistan-and-bangladesh/
13. sigma/rules/windows/dns_query/dns_query_win_mal_cobaltstrike.yml at master - GitHub, accessed April 4, 2026, https://github.com/SigmaHQ/sigma/blob/master/rules/windows/dns_query/dns_query_win_mal_cobaltstrike.yml
14. 2025 Year in Review: Malicious, Infrastructure - Recorded Future, accessed April 4, 2026, https://www.recordedfuture.com/research/2025-year-in-review-malicious-infrastructure
15. CrossC2 Expanding Cobalt Strike Beacon to Cross-Platform Attacks - JPCERT/CC Eyes, accessed April 4, 2026, https://blogs.jpcert.or.jp/en/2025/08/crossc2.html
16. Your BOFs Are gross, Put on a Mask: How to Hide Beacon During BOF Execution | IBM, accessed April 4, 2026, https://www.ibm.com/think/x-force/how-to-hide-beacon-during-bof-execution
17. Engineering a Highly Customized Malleable C2 Profile | by Khaled ..., accessed April 4, 2026, https://khaled0x07.medium.com/engineering-a-highly-customized-malleable-c2-profile-30e0efee307c
18. The CyberThreat Report: October 2025 - Trellix, accessed April 4, 2026, https://www.trellix.com/advanced-research-center/threat-reports/october-2025/
19. Harnessing the Power of Cobalt Strike Profiles for EDR Evasion ..., accessed April 4, 2026, https://whiteknightlabs.com/2025/05/19/harnessing-the-power-of-cobalt-strike-

profiles-for-edr-evasion-part-2/
20. sigma/rules/windows/process_creation/proc_creation_win_hktl_cobaltstrike_load_by_rundll32.yml at master - GitHub, accessed April 4, 2026, https://github.com/SigmaHQ/sigma/blob/master/rules/windows/process_creation/proc_creation_win_hktl_cobaltstrike_load_by_rundll32.yml
21. JA4+ Fingerprinting: The new weapon for analysts, now integrated ..., accessed April 4, 2026, https://strangebee.com/blog/ja4-fingerprinting-now-available-in-thehive/
22. JA4+ Network Fingerprinting - FoxIO Blog, accessed April 4, 2026, https://blog.foxio.io/ja4+-network-fingerprinting
23. Cobalt Strike: A Cyber Assessment Challenge | ITEA Journal, accessed April 4, 2026, https://itea.org/journals/volume-45-3/cobalt-strike-cyber-assessment-challenge/
24. Release Upgrade Summary (2025) - Zscaler Help Portal, accessed April 4, 2026, https://help.zscaler.com/zia/release-upgrade-summary-2025?applicable_category=zscloud.net&deployment_date=2025-05-12&id=1508871
25. 8 Cyber Predictions for 2025: A CSO's Perspective - Zscaler, Inc., accessed April 4, 2026, https://www.zscaler.com/blogs/security-research/top-cyber-predictions-2025-cso-perspective
26. Building a SOC Detection Engineering Lab with Elastic, Mythic C2, and Attack Simulations, accessed April 4, 2026, https://medium.com/@gregd03/building-a-soc-detection-engineering-lab-with-elastic-mythic-c2-and-attack-simulations-3e9200b41cd8
27. Best Practices Guide for Detection Engineering in Elastic SIEM, accessed April 4, 2026, https://www.cybernx.com/detection-engineering-in-elastic-siem-guide/
28. Elastic Security's Detection Engineering Capabilities: A Comprehensive Guide - ITButler, accessed April 4, 2026, https://itbutler.sa/blog/elastic-securitys-detection-engineering-capabilities-a-comprehensive-guide/
29. Threat Actor Profile: Interlock Ransomware - Arctic Wolf, accessed April 4, 2026, https://arcticwolf.com/resources/blog/threat-actor-profile-interlock-ransomware/
30. CXO Monthly Roundup, January 2026: Zscaler ThreatLabz AI Security Report, new APT campaigns targeting government entities, technical analyses of the BlackForce phishing kit and Matanbuchus 3.0, and more, accessed April 4, 2026, https://www.zscaler.com/cxorevolutionaries/insights/cxo-monthly-roundup-january-2026-zscaler-threatlabz-ai-security-report-new-apt-campaigns
31. sigma/rules/windows/builtin/system/service_control_manager/win_system_cobaltstrike_service_installs.yml at master - GitHub, accessed April 4, 2026, https://github.com/SigmaHQ/sigma/blob/master/rules/windows/builtin/system/service_control_manager/win_system_cobaltstrike_service_installs.yml
32. Sliver C2 Deep Dive : A Comprehensive Command & Control Framework Series - Medium, accessed April 4, 2026, https://medium.com/@maverickcx64/sliver-c2-deep-dive-a-comprehensive-command-control-framework-series-4f8ba55f7a45

33. The Best Red Teaming Tools of 2026: What You Need to Know - IT Security Guru, accessed April 4, 2026, https://www.itsecurityguru.org/2025/12/11/the-best-red-teaming-tools-of-2026-what-you-need-to-know/
34. Sliver, Havoc, and Mythic: Complementary C2 Frameworks - CWL : Advanced Cyber Attack & Detection Learning Platform, accessed April 4, 2026, https://cyberwarfare.live/sliver-havoc-and-mythic-complementary-c2-frameworks/
35. C2 Frameworks - Red Canary Threat Detection Report, accessed April 4, 2026, https://redcanary.com/threat-detection-report/trends/c2-frameworks/
36. Release Upgrade Summary (2025) - Zscaler Help Portal, accessed April 4, 2026, https://help.zscaler.com/deception/release-upgrade-summary-2025
37. H1 2025 Malware and Vulnerability Trends - Recorded Future, accessed April 4, 2026, https://www.recordedfuture.com/research/h1-2025-malware-and-vulnerability-trends